



ELSEVIER

Available online at www.sciencedirect.com

SCIENCE @ DIRECT®

Computers & Operations Research 31 (2004) 2315–2332

computers &
operations
research

www.elsevier.com/locate/dsw

A method for solving the minimization of the maximum number of open stacks problem within a cutting process

José Carlos Becceneri^{a,*}, Horacio Hideki Yanasse^b, Nei Yoshihiro Soma^c

^a*Laboratory for Computing and Applied Mathematics, Brazilian Space Research Institute, P.O. Box 515-INPE/LAC, São José dos Campos, SP, 12201-970, Brazil*

^b*Special Technologies Center, Brazilian Space Research Institute, P.O. Box 515-INPE/CTE, São José dos Campos, SP, 12201-970, Brazil*

^c*Aeronautic Institute of Technology, Aerospace Technical Center CTA/ITA/IEC, Pça. Mal. Eduardo Gomes, 50, Vila das Acácias, São José dos Campos, SP, 12228-900, Brazil*

Abstract

In this paper, the problem of minimizing the maximum number of open stacks around a saw machine is addressed. A new heuristic and a branch-and-bound based exact method for the problem are presented. Computational tests comparing the new approach with other algorithms previously suggested in the literature were carried out.

© 2003 Elsevier Ltd. All rights reserved.

Keywords: Cutting stock; Pattern sequencing; Graph traversing; Branch-and-bound

1. Introduction

The problem addressed here arises in production environments where one is faced with the problem of cutting smaller pieces from larger objects to satisfy clients' orders. The pieces formed during the raw material cutting process form stacks around a saw machine. Equal pieces are heaped together in a single open stack until their demand is satisfied; thereafter the resulting stack is removed from the vicinity of the machine.

In the production systems studied in this paper, we perform task sequencing (cutting patterns) to minimize the maximum number of open stacks during the cutting process, which is carried

* Corresponding author.

E-mail addresses: becce@lac.inpe.br (J.C. Becceneri), horacio@lac.inpe.br (H.H. Yanasse), nysoma@comp.ita.cta.br (N.Y. Soma).

out by a single saw machine. This minimization process implies in an efficient utilization of scarce resources, since it is reasonable to suppose that too many stacks cannot remain opened simultaneously due to physical space limitations. For instance, some unfinished stacks may have to be removed temporarily, to allow new stacks to be opened. Those removed stacks are brought back again to the saw machine vicinity whenever the pieces corresponding to them are to be cut and by maintaining the smallest possible number of unfinished stacks (ξ) the associated costs for such handling can also be minimized. The problem of determining ξ and the pattern sequence that generates it is known as the minimization of open stack problem (MOSP). A different scenario where this problem appears is in the VLSI design, more specifically in the gate matrix layout problem. Further industrial applications are presented in [1].

In this work, methods for solving this cut pattern-sequencing problem are presented and, the contributions include a new heuristic, and improvements in an exact method (branch-and-bound).

The next sections are organized as follows: in Section 2 we present a brief summary of earlier works on cut pattern sequencing, in Section 3 we show how simple pre-processing procedures on the input data can simplify the problem, in Section 4 a new heuristic is developed and in Section 5 we present the improvements introduced in a branch-and-bound method and we give details of an implementation of a lower bound which appeared previously in the literature. In Section 6, the computational experiments are reported and conclusions and future works are presented in Section 7.

2. Previous works

In [2] a cut pattern sequence problem is studied aiming at reducing the piece discontinuity numbers, i.e. if a given piece is not present in two subsequent cutting patterns a discontinuity exists. In [3] a procedure to reduce the cut distance among equal pieces is presented and, in [4] the objective is to reduce the spread of the user's order, a spread is the quantity of different processed patterns between the cut of the first and the last piece. In [5] a limited version of the MOSP is introduced, where any pattern has to be formed by two different pieces only. In [6] a meta-heuristic approach to solve the MOSP and the order spread minimization problem is introduced. In [7] three heuristics to solve the order spread minimization problem are analyzed.

In [8,9] six heuristics for solving the MOSP are presented and the so-called Yuen3 (third heuristic) is indicated (and usually accepted) as the one, which has a very good practical behavior, both in processing time as well as in simplicity of implementation, cf. [10]. The Yuen3 was also tested against the algorithms introduced here. In [11] the six heuristics of [8,9] are fully explained and some optimality conditions for the problem are presented. In [10] a two-phase approach to solve the MOSP is given, in the first phase, a solution is found by a heuristic procedure and in the second phase, an enumeration on the patterns' set is made using a branch-and-bound method.

The works [1,12,13] form the foundations to the solution procedures presented here and the major ideas will be briefly stated in Sections 4 and 5. A linear integer programming (LIP) based formulation to the MOSP is presented in [14], but in [15] there is an allusion that a direct attack via LIP may be unfeasible in practice. This last hypothesis is reinforced by the fact that the problem has been proved to be NP-Hard (see [16]).

3. Pre-processing

It is assumed, without loss of generality, that the input data for an instance of the MOSP is given by a matrix of patterns (rows) and pieces (columns). The sole information about the patterns that are relevant for the MOSP are the pieces types contained in them. Hence, if the set of all piece types is $p = \{p_1, \dots, p_m\}$ and the given patterns are denoted by $P_1, \dots, P_v$, then, a pattern P_i in our problem can be represented solely as a set of piece types $\{P_{iu}, \dots, P_{ij}, \dots, P_{ik}\}$, with $P_{it} \in p$, $t = u, \dots, j, \dots, k$, and the input data can be presented as a $v \times m$ matrix of 0's and 1's where in row i , corresponding to pattern P_i , there will be 1's in columns $u, \dots, j, \dots, k$ in correspondence to the piece types $P_{iu}, \dots, P_{ij}, \dots, P_{ik}$ present in pattern P_i , and 0's in the remaining columns.

The input data for an instance of the MOSP can also be given by a graph $G(V, E)$. In $G(V, E)$ (from now on denoted simply as G), node $k \in V$ represents the piece type p_k and two nodes i and j are adjacent (that is, $(i, j) \in E$) if and only if the corresponding piece types p_i and p_j are simultaneously present in some pattern. A pattern with h different piece types defines a clique of size h (K_h) and a feasible solution to the MOSP is associated to a sequence of arcs of G . Moreover, an optimal solution to the problem can also be viewed as a succession of (probably) large cliques.

The pre-processing phase has two main objectives, (i) to eliminate redundancies and (ii) to detect conditions for which a solution can be found in polynomial time.

A redundancy occurs whenever in G , a clique is an induced complete sub-graph of another possibly larger one. This case happens whenever $P_j \subseteq P_i$, $i \neq j$ and we say that pattern P_i dominates pattern P_j . In such a case, P_j can be removed during an optimal solution search of the problem, since P_j can be inserted immediately after the pattern P_i in an optimal sequence, without increasing the value of ξ . It is clear that when $P_j \not\subseteq P_i$ no such a dominance can exist, even if $|P_i| \geq |P_j|$, P_i and P_j share an associated non-maximal clique of size smaller than $|P_j|$, i.e. if $K_s \subseteq K_{P_i}$, $K_s \subseteq K_{P_j}$ and $|P_i| \geq |P_j| > s$.

After the dominance analysis among patterns, in a subsequent phase, we verify if some induced sub-graph of G has trees, simple cycles or 1-trees as components. If this is the case, we apply then the algorithms developed in [1] to generate an optimal ordering for the pieces contained in these components. The central idea is to explore the structure of graphs which possess “small” cliques, e.g. if G is a polygon then the MOSP is solved in linear time and $\xi = 2$. In general, an optimal solution to the MOSP is given by a sequence of possibly large and overlapping cliques, intertwined by elements such as trees. Therefore, it is important to exactly solve the schedule of the latter components in polynomial time to incorporate them into the final solution. After the pre-processing phase, we will work with the minor of G , which will be denoted by G_p onwards.

4. The heuristic of Minimal Cost Node

We present next a Minimal Cost Node heuristic that uses the idea of traversing the arcs in G_p , using as criterion, the least quantity of arcs to be sequenced in order to close the node.

The order in which the arcs are traversed dictates the order in which the patterns are sequenced.

Let us denote:

- V is a set of n nodes (associated to the pieces) of G_p ;
- E is the set of nodes of G_p ;

- $\Omega(k)$ is the degree of the node $k \in V$ excluding the arcs incident to k already traversed;
- SETV is an ordered set of nodes of V yet to be sequenced. The nodes are ordered in a non-decreasing manner according to Ω . A node $k \in V$ is in SETV if and only if $\Omega(k) \geq 1$;
- ARC is an ordered set of arcs of G_p already traversed;
- OPEN is the set of nodes in V already open (at least one arc incident to this node is in set ARC and not all arcs incident to this node is in the set ARC);
- s is the number of arcs in ARC ($|\text{ARC}|$);
- ξ' is the maximum number of open stacks generated by the heuristic.

We introduce in the sequence a pseudo-code of the heuristic of Minimal Cost Node.

Heuristic of Minimal Cost Node

Input: Graph G_p

Output: Sequence of patterns to be cut and ξ'

Build SETV;

OPEN $\leftarrow \emptyset$;

ARC $\leftarrow \emptyset$;

$s \leftarrow 0$;

While $s \neq |E|$

Begin

 get the degree $\Omega(k)$ of the first node k of SETV.

 get the first two adjacent nodes of SETV (n_1 and n_2) with

$\Omega(n_1) = \Omega(k)$ and that possess pair-wise the smallest measures Ω .

 put n_1 and n_2 in OPEN (if they are not there);

 get the arc (n_1, n_2) and insert it in ARC;

$s \leftarrow s + 1$;

 update $\Omega(n_1)$ and $\Omega(n_2)$;

If $(\Omega(n_k) = 0) \ // \ k = 1, 2$

Then Delete n_k from OPEN

 search the nodes in OPEN trying to find adjacent nodes that have arcs not yet included in ARC.

If there are such arcs

Then Begin

 put all of them in ARC;

 update s ;

 update the $\Omega(n_k)$ for all end-nodes of the arcs inserted in ARC;

 update OPEN;

 update SETV;

End;

End;

With the ordered set ARC generate the sequence of patterns from G_p and G to obtain ξ' .

Table 1
Matrix of patterns $\times$ pieces

Pattern	Piece							
	p_1	p_2	p_3	p_4	p_5	p_6	p_7	p_8
P_1	1	1	1	0	0	0	0	0
P_2	1	0	0	1	0	0	0	0
P_3	1	0	0	0	1	0	0	0
P_4	1	0	0	0	0	1	0	0
P_5	1	0	0	0	0	0	1	0
P_6	1	0	0	0	0	0	0	1
P_7	0	1	0	0	1	0	0	0
P_8	0	1	0	0	0	0	1	0
P_9	0	0	1	0	1	0	0	0
P_{10}	0	0	0	1	0	1	0	0
P_{11}	0	0	0	1	0	0	0	1
P_{12}	0	0	0	0	1	1	0	0
P_{13}	0	0	0	0	1	0	1	0
P_{14}	0	0	0	0	0	1	0	1

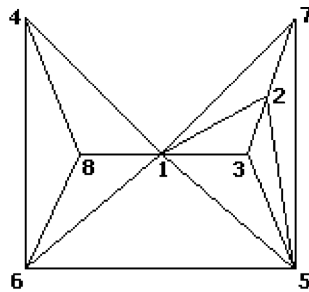


Fig. 1. Graph G_p for the example.

For the sake of clarity we illustrate the heuristic with a numerical example. Consider an instance with initial matrix of patterns (lines) by pieces (columns) given in Table 1.

The corresponding MOSP graph (G) is given in Fig. 1. Notice that parallel arcs, without loss of generality, are treated as a single one, since once the first arc is traversed, all the parallel ones can be immediately traversed for both nodes are already open.

The main solution steps performed by the Minimal Cost Node heuristic are presented next.

There is no dominated pattern in this instance ($G_p = G$). The program executes seven loops to finish. Next we present just four of the loops.

Initial condition:

$\Omega(1)$	$\Omega(2)$	$\Omega(3)$	$\Omega(4)$	$\Omega(5)$	$\Omega(6)$	$\Omega(7)$	$\Omega(8)$
7	4	3	3	5	4	3	3

OPEN = { } and ARC = { }
 SETV = {3, 4, 7, 8, 2, 6, 5, 1}

ξ'	s	n_1	n_2
0	0	—	—

First loop:

$\Omega(1)$	$\Omega(2)$	$\Omega(3)$	$\Omega(4)$	$\Omega(5)$	$\Omega(6)$	$\Omega(7)$	$\Omega(8)$
7	4	3	2	5	4	3	2

OPEN = {4, 8} and ARC = {(4,8)}
 SETV = {4, 8, 3, 7, 2, 6, 5, 1}

ξ'	s	n_1	n_2
2	1	4	8

Second loop:

$\Omega(1)$	$\Omega(2)$	$\Omega(3)$	$\Omega(4)$	$\Omega(5)$	$\Omega(6)$	$\Omega(7)$	$\Omega(8)$
7	4	3	1	5	2	3	1

OPEN = {4, 6, 8} and ARC = {(4,8), (4,6), (6,8)}
 SETV = {4, 8, 6, 3, 7, 2, 5, 1}

ξ'	s	n_1	n_2
3	3	4	6

Third loop:

$\Omega(1)$	$\Omega(2)$	$\Omega(3)$	$\Omega(4)$	$\Omega(5)$	$\Omega(6)$	$\Omega(7)$	$\Omega(8)$
4	4	3	0	5	1	3	0

OPEN = {1, 6} and ARC = {(4,8), (4,6), (6,8), (1,4), (1,6), (1,8)}
 SETV = {6, 3, 7, 1, 2, 5}

ξ'	s	n_1	n_2
4	6	4	1

	p ₁	p ₂	p ₃	p ₄	p ₅	p ₆	p ₇	p ₈	# open stacks
P ₁₁				•				•	2
P ₁₀				•		•		•	3
P ₁₄				•		•		•	3
P ₂	•			•		•		•	4
P ₄	•					•		•	3
P ₆	•					•		•	3
P ₁₂	•				•	•			3
P ₃	•				•				2
P ₉	•		•		•				3
P ₁	•	•	•		•				4
P ₇	•	•			•				3
P ₅	•	•			•		•		4
P ₈		•			•		•		3
P ₁₃					•		•		2

Fig. 2. Open stacks generated (•) during the processing of the patterns.

Final loop:

$\Omega(1)$	$\Omega(2)$	$\Omega(3)$	$\Omega(4)$	$\Omega(5)$	$\Omega(6)$	$\Omega(7)$	$\Omega(8)$
0	0	0	0	0	0	0	0

OPEN = { } and ARC = {(4,8), (4,6), (6,8), (1,4), (1,6), (1,8), (6,5), (1,5), (1,3), (3,5), (3,2), (1,2), (2,5), (1,7), (2,7), (5,7)}

SETV = { }

ξ'	s	n_1	n_2
4	16	1	7

The sequence of patterns to be cut is dictated according to the ordered set ARC. In [12], the procedure to transform a sequence of arcs in a sequence of patterns is presented. The idea is the following: “follow the sequence of the arcs and sequence a pattern P_i when, for the first time, all the nodes corresponding to all the piece types in P_i are open”. Hence, the final pattern sequence obtained for this instance is $\{P_{11}, P_{10}, P_{14}, P_2, P_4, P_6, P_{12}, P_3, P_9, P_1, P_7, P_5, P_8, P_{13}\}$.

The open stacks obtained during the processing of the patterns using this sequence are presented in Fig. 2.

5. The exact method

In this section, we present a branch-and-bound approach for exactly solving the MOSP. The search for an optimal solution follows a depth first strategy with a local dominance scheme as described in

[13,14], which generated shortcuts in the enumeration of feasible solutions schema. This dominance appears from the observation that, if, in a given level, the sequence of pieces p_i, p_j is being already considered, therefore the minimum number of open stacks found so far cannot be diminished by the pieces p_i, p_j relative order reversal. Additionally, a further improvement was incorporated into the algorithm by using the concept of dominance of nodes.

Let $\underline{N}_b(i)$ be the “neighborhood” of a node i , that is, is the set of nodes that are adjacent to i , and $\underline{N}_b(i) = \underline{N}_b(i) \cup \{i\}$ in G_p . The next proposition provides a possible ordering that can be assumed in an optimal sequence of pieces whenever dominance of nodes is identified.

Global dominance proposition. *If G_p has two (different) nodes, i and j and $\underline{N}_b(j) \subseteq \underline{N}_b(i)$, then associated to any feasible solution of the problem, it is always possible to construct a new sequence, whose maximum number of open stacks is less than or equal to the original sequence and the stack associated to piece p_j is closed before that of p_i .*

Proof. Let us admit a given closing sequence of stacks S , where $\underline{N}_b(j) \subseteq \underline{N}_b(i)$. If in that sequence j appears before i then, clearly, nothing needs to be done. Otherwise, if j appears after i , then when sequencing i , all the stacks that also close j are opened. Therefore, we can sequence j before i , without increasing the total number of open stacks implying that the number of stacks of the new sequence will be smaller than or equal to that generated by S .

Applying the result of the previous theorem into the search for an optimal solution, whenever $\underline{N}_b(j) \subseteq \underline{N}_b(i)$, it is sufficient to analyze sequences where j appears before i (the stack of piece j is closed before the stack of piece i). Notice that in spite of the fact that this theorem generalizes the idea of dominance, it was not incorporated into the exact algorithm, since it worsened the overall performance observed in the computational experiments. An attempt to explain such behavior is perhaps the fact that there can exist many optimal solutions where the stack of p_j is completed after the stack of p_i and, these solutions are ignored if this theorem is applied during the enumeration. The computational effort in selecting only sequences that obey the theorem does not compensate the savings in the enumeration.

The improvement on the enumeration scheme is obtained by the identification of equivalent solutions via permutation analysis and it can be directly implemented based on the following proposition.

Equivalency proposition. *If G_p has two (different) nodes, i and j with $\underline{N}_b(i) = \underline{N}_b(j)$ or $\underline{N}_b(i) = \underline{N}_b(j)$, then from a given feasible solution of the problem where p_i is not immediately followed by p_j or vice-versa, it is possible to construct a new sequence with p_i and p_j appearing consecutively. Moreover, the maximum number of open stacks by the new sequence is less than or equal to the original sequence.*

Proof. Let us assume without loss of generality that $1, \dots, i, \dots, j, \dots, n$, $i \neq j$ is a feasible sequence of the n vertices of G_p , and $\underline{N}_b(i) = \underline{N}_b(j)$. When we sequence all the incident arcs to i , the set of open stacks associated to its neighboring vertices is the same set that would be opened by p_j . Therefore, it is possible to sequence j immediately before i , since the total number of open stacks remains the same. A similar argument can be used to prove the case when $\underline{N}_b(i) = \underline{N}_b(j)$.

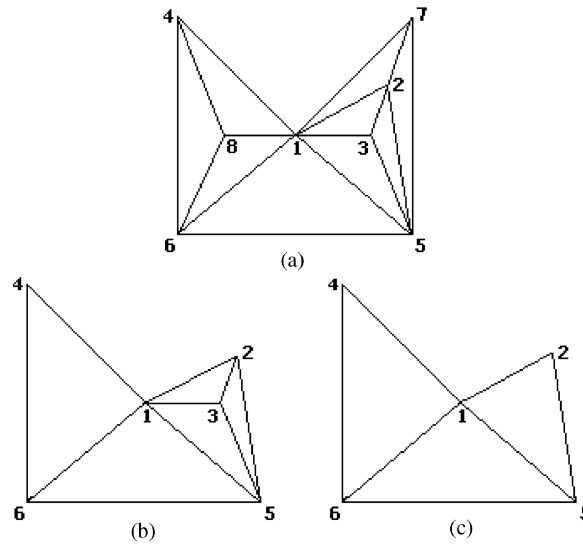


Fig. 3. Identification of equivalencies.

We can conclude that if in G_p nodes i and j satisfy the given conditions, for the sake of finding an optimal sequencing, we can delete node i (or node j) and all arcs incident to it.

In Fig. 3, part a, an initial G_p is shown. The reduction process is applied until its exhaustion. G_p is reduced, firstly to the graph indicated in part b of Fig. 3, because there are equivalency between the nodes 3 and 7, 4 and 8. Analysing the resultant graph, we see a new equivalence (between the nodes 2 and 3). Applying, again the reduction method, we get a new graph (Fig. 3, part c). The order of G_p decreases from 8 to 5, while the quantity of arcs varies from 15 to 7. Observe that there is a level of hierarchy on the equivalence relations that must be preserved. Nodes 2 and 3 are equivalent only after the first iteration; they were not equivalent in the first iteration. Therefore, we must keep in mind that nodes 3 and 2 can be sequenced one immediately after the other but node 3 must precede node 2 in the sequence.

We next recall three lower bounds for the problem since they are used in the exact method.

- $lb_p = \text{Max} \{|P_j|, j = 1, \dots, n\}$,
- $lb_d = 1 + \text{Min}\{\text{degree}(v) | v \text{ is a vertex of } G_p\}$,
- $lb_c = \text{Max}\{K_1 | K_1 \text{ is a minor of } G_p\}$.

The first two bounds are immediate, since lb_p says that ξ cannot be smaller than the maximum number of pieces of any given pattern. lb_d is derived from the fact that a vertex v , of minimal degree in G_p will have open stacks associated with p_v and all those p_j 's having p_v and p_j belonging to the same pattern. Since some vertex has to be sequenced first at least this number of stacks will be opened.

The third bound lb_c , stated in [17], is also simple. It indicates that a lower bound is obtained if a minor of G_p is a complete graph. To determine the best lower bound the following procedure is suggested.

Heuristic Arc Contraction**Input:** G_p **Output:** lb_c

Step 0: // Initialization
 $lb \leftarrow \text{Max}\{lb_p, lb_d\};$
 $G \leftarrow G_p;$

Step 1:
If G is a complete graph
 Then Begin
 $lb_c = lb;$
 Halt;
 End;
If $|G| \leq lb$ // $|G|$ is the order of G
 Then Begin
 $lb_c = lb;$
 Halt;
 End;
 Else Let S be the vertex set of G sorted in non-decreasing order;

Step 2:
 Choose an arc a from G such that their ends have the smallest indices in S ;
 Contract arc a ; // $G \leftarrow \text{Minor}(G);$
 $d_{\min} = \text{minimum degree of } G;$
 If ($d_{\min} \geq 2$ **and** $d_{\min} + 1 > lb$) $lb = d_{\min} + 1;$
 return to Step 1;

It is worth of mentioning that the set S in the Arc Contraction heuristic needs to be updated at each iteration to guarantee that the lower bound is as large as possible. For instance, for the graph G given in Fig. 4, which is three-regular, if set S is sorted just once, two different cliques, K_3 and K_4 , could be obtained, cf. respectively, Figs. 5 and 6.

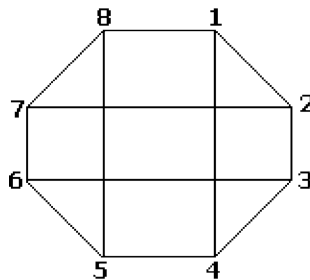


Fig. 4. Initial graph $G(\equiv G_p)$.

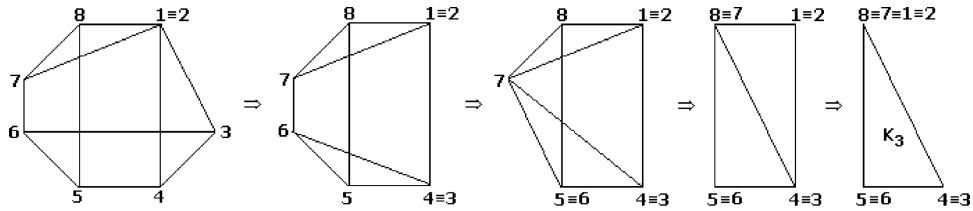


Fig. 5. Minor of G_p generating K_3 .

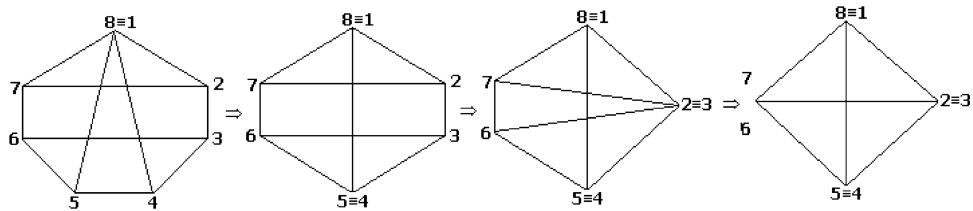


Fig. 6. Minor of G_p generating K_4 .

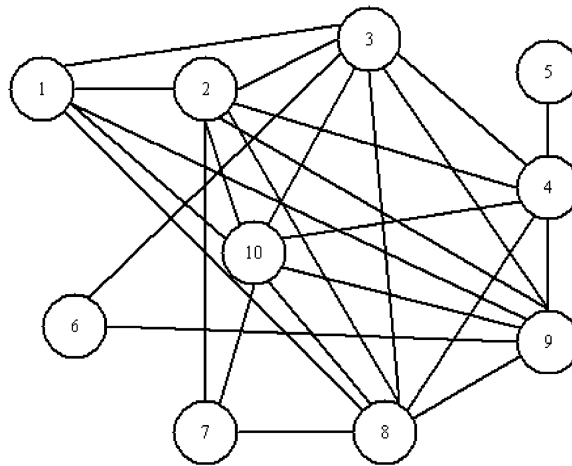


Fig. 7. Large deviations amidst the lower bounds.

The value obtained using lb_c can be much better than the values obtained using lb_p or lb_d . Consider the following instance of a MOSP: $P_1 = \{p_1, p_8\}$, $P_2 = \{p_2, p_7\}$, $P_3 = \{p_3, p_6\}$, $P_4 = \{p_4, p_6\}$, $P_5 = \{p_4, p_5\}$, $P_6 = \{p_6, p_9\}$, $P_7 = \{p_7, p_8\}$, $P_8 = \{p_7, p_{16}\}$, $P_9 = \{p_2, p_3\}$, $P_{10} = \{p_1, p_2, p_3\}$, $P_{11} = \{p_2, p_3, p_4\}$, $P_{12} = \{p_1, p_8, p_9\}$, $P_{13} = \{p_2, p_8, p_9\}$, $P_{14} = \{p_3, p_8, p_9\}$, $P_{15} = \{p_4, p_8, p_9\}$, $P_{16} = \{p_1, p_{10}\}$, $P_{17} = \{p_2, p_{10}\}$, $P_{18} = \{p_3, p_{10}\}$, $P_{19} = \{p_4, p_{10}\}$, $P_{20} = \{p_8, p_9, p_{10}\}$. The associated graph G_p is given in Fig. 7 and the values for the lower bounds are $lb_p = 3$, $lb_d = 2$ and $lb_c = 6$. So, for this instance, the improvement observed is quite impressive.

We present next the major steps of an exact method for solving MOSP that takes into account the observations presented previously.

Exact Method: branch and bound approach

- Step 0:** Determine graph G , where all components that are trees are removed, since they shall be considered at the end, call this removed set as D .
- Step 1:** Execute the pre-processing on G to obtain G_p . //Section 3
- Step 2:** Execute the Heuristic of Minimal Cost Node to obtain ξ' and the sequence of pieces that generates it;
- Step 3:** Execute the Heuristic Arc Contraction to get the lower bound lb_c ;
- Step 4:** **If** $\xi' = lb_c$ //upper bound = lower bound
 Then the optimal solution was found in Step 2
 Else Begin
 Construct the equivalence set E
 If $E \neq \emptyset$
 Then Begin
 Construct a new graph G_p by deleting the equivalent nodes;
 Call Heuristic Arc Contraction; // to determine a new lb_c
 End;
 Execute a branch-and-bound in order to obtain an optimal solution;
 Insert the deleted equivalent nodes;
 End;
- Step 5:** Insert the vertices of D into the final sequence.

In practice, the algorithm can take a long time to solve larger sizes instances of the problem, therefore, a time limit may be imposed and the best solution found so far is returned.

6. Implementation and computational tests

All the software developed was written in C++, with the Microsoft Visual C++ 6.0, using the Windows 98 platform and a Pentium II-400, with 128 MB of RAM.

Several programs were developed to generate data for the tests. A matrix of 0's and 1's is generated with a pre-established value for the maximum quantity of ones that each row can have. Another program to generate MOSP graphs was implemented. We start with a complete graph and then a percentage of arcs is randomly deleted, assuring that a disconnected graph will not be generated. In this case, the number of nodes and the percentage of arcs to be deleted are pre-defined.

The computational tests were performed to evaluate the heuristic and the improvement introduced on the exact method.

The Smaller Cost Node heuristic and Yuen3 were compared with lower bounds lb_p and lb_c . The results of these tests are shown in Table 2.

In Table 2, in column $n \times m$ the numbers of patterns and pieces of the test cases are indicated, in column C the maximum quantity of ones that any line can hold is indicated, lb_c and lb_p are the

Table 2
Comparison between Smaller Cost Node heuristic and Yuen3

$n \times m$	C	lb_p	lb_c	t	tY	MNOS	MNOSY
10 × 10	5	4.9	6.2	0.18	0.01	6.6	7.6
10 × 10	6	6.0	7.6	0.14	0.01	7.9	8.5
10 × 10	7	7.0	8.6	0.20	0.01	8.6	9.2
10 × 10	8	7.4	8.8	0.14	0.01	9.0	9.2
20 × 20	6	6.0	10.3	0.32	0.01	11.6	13.7
20 × 20	8	7.9	12.9	0.40	0.01	15.3	16.2
20 × 20	10	10.0	15.8	0.39	0.01	16.3	17.8
20 × 20	12	11.8	17.3	0.61	0.01	17.7	19.4
20 × 20	16	15.8	19.8	0.38	0.01	19.8	20
30 × 30	8	8	15.7	0.99	0.01	20.0	22.8
30 × 30	12	12	22.7	1.32	0.01	25.4	28.3
30 × 30	16	15.8	26.9	1.65	0.01	27.8	30.0
30 × 30	20	19.9	29.2	1.25	0.01	29.4	30.0
40 × 40	10	10	22.5	0.65	0.01	29.0	34.2
40 × 40	15	15	31.5	1.31	0.01	34.9	38.3
40 × 40	20	20	36.7	2.32	0.01	37.8	39.8
50 × 50	10	10	26.0	1.35	0.01	35.6	41.6
50 × 50	15	15	36.4	2.58	0.01	42.7	49.0
50 × 50	20	20	43.1	4.02	0.01	45.6	50.0
50 × 50	30	30	49.3	8.03	0.01	49.3	50.0
60 × 60	15	15	39.2	4.13	0.01	50.2	57.8
60 × 60	20	20	50.4	7.51	0.01	55.4	59.8
60 × 60	25	25	55.2	10.34	0.01	56.8	60
100 × 100	15	15	50.9	18.82	0.02	81.7	95.5
100 × 100	20	20	66.8	31.21	0.02	88.5	98.2
100 × 100	30	30	87.8	73.19	0.02	95.4	100
100 × 100	40	40	97.1	138.74	0.02	97.9	100
150 × 150	15	15	60	56.82	0.07	119.4	139.0
150 × 150	20	20	82.8	106.14	0.07	131.4	149
150 × 150	25	25	103.4	182.51	0.07	138.0	150
150 × 150	30	30	118.3	302.49	0.09	142.2	150

lower bounds as explained in Section 5, in column t the average time in seconds to process the heuristic (that is, reading the data, generation of the graph, calculation of the bounds and generation of a solution) is given, in column MNOS the average maximum values obtained with the Smaller Cost Node heuristic is given. Columns tY and MNOSY are the corresponding values of t and MNOS using Yuen3. For each line of the table, 20 test cases were executed.

As can be seen, the solution obtained using the Smaller Cost Node heuristic is better than the solution obtained using Yuen3 heuristic, at the expense of a much larger computational effort. However, the computational times are not prohibitive. All problems tested, up to 150 patterns and 150 different pieces took, on average, less than 5 min of CPU time.

With the determined lower bounds, the quality of the solutions can be assessed. As can be seen, for relative larger values of C (relative to the total number of pieces in the instances), the quality

Table 3

Performance of the heuristic as a function of the parameter C

Number of nodes	C	$\Delta_{\text{NH}} = 0$	$\Delta_{\text{NH}} = 1$	$\Delta_{\text{NH}} = 2$	$\Delta_{\text{NH}} \geq 3$	Δ_{lb}
20	8	6	4	0	0	3
20	12	10	0	0	0	1
20	16	10	0	0	0	0
30	8	6	4	0	0	5
30	12	8	2	0	0	3
30	16	8	2	0	0	1
40	10	6	3	1	0	8
40	15	8	2	0	0	4
40	20	10	0	0	0	1
60	15	9	1	0	0	12
60	20	8	2	0	0	7
60	25	10	0	0	0	3

of the solution obtained by the new heuristic is quite good. In two sets of instances (20×20) with $C = 16$ and (50×50) with $C = 30$, all instances were solved to optimality by the heuristic. When the lower bound obtained is compared with the solution obtained by the heuristic for smaller relative values of C , the gap is (relatively) large. So the following possibilities may be happening: (i) the quality of the lower bound is not good; (ii) the heuristic is not performing well or (iii) both (i) and (ii) are happening. In order to verify what is actually happening, some instances were solved to optimality to compare with the heuristic solution and lower bounds. From the limited tests performed, it seems that the quality of the lower bound deteriorates for relative small values of C ; the quality of the solution obtained by the heuristic seems to deteriorate a bit but still remains quite good. In Table 3, Δ_{NH} represents the difference of the quantity of open stacks determined by the Smaller Cost Node heuristic and the best solution (with a maximum of 1 h of CPU time), found by the optimized branch-and-bound method. In column $\Delta_{\text{NH}} = 0$ the number of instances where the Smaller Cost Node Heuristic found the optimal solution is indicated, in columns $\Delta_{\text{NH}} = 1$, $\Delta_{\text{NH}} = 2$ and $\Delta_{\text{NH}} \geq 3$ the number of instances where the difference between the two solutions is 1 stack, 2 stacks and 3 or more stacks are indicated respectively; C is the maximum quantity of ones that any row can have, Δ_{lb} is the difference between lb_c and the best solution. The results in each line correspond to 10 instances tested.

From the tests performed, we can verify that the quality of the solution obtained by the Smaller Cost Node heuristic was quite good. The quality of the lower bound, on the other hand, was quite poor in some instances.

It may be worth observing that the Arc Contraction heuristic may produce different bounds depending on the nodes chosen for the arc contraction. In computational tests we performed, by simply exchanging some columns of the input matrix of a few instances, we observed slight differences on the results.

Several computational tests were also performed to compare the obtained gain with the improvements suggested here, in the branch-and-bound method. The results are summarized in Table 4. The time limit was set to 3600 s.

Table 4
Comparison between optimized and non-optimized branch-and-bound

Id	lb _c	NH	P1	t1	N1	P2	t2	N2
G302501	21	25	24	1.1	70(36)	24	7.8	663(143)
02	20	25	25	1.5	55	25	14.7	1002
03	20	25	24	1.2	76(40)	24	11.9	858(201)
04	20	24	24	0.7	39	24	9.5	686
G304001	17	22	22	5.99	167	22	127.0	5592(222)
02	17	23	22	4.9	160(32)	22	64.4	3436(82)
03	17	23	23	8.56	229	23	136.1	6311
04	16	23	23	7.63	181	23	112.0	5072
G306001	13	20	19	45.44	899(52)	19	1444.5	53934(382)
02	12	18	18	20.7	497	18	910.7	29954
03	12	20	19	42.96	872(30)	19	3600	66022(30)
04	13	19	19	53.17	1063	19	3551.6	101785
G402501	27	34	34	15.49	97	34	144.0	2393
02	27	34	34	14.72	121	34	230.2	3338
03	26	34	34	15.48	115	34	240.7	3293
04	27	35	34	18.46	165(62)	34	246.9	3488(533)
G404001	22	31	31	58.83	329	31	1216.7	13392
02	22	32	31	61.22	369(172)	31	998.7	11181(3680)
03	22	31	31	68.71	357	31	1966.4	17444
04	22	32	31	61.17	377(42)	31	1390.3	14802(185)
G406001	16	27	27	1031.7	3295	27	3600	17281
02	16	27	26	669.1	2476(436)	27	3600	19248
03	17	27	26	848.7	2753(44)	26	3600	18653(138)
04	16	28	27	738.9	2605(70)	27	3600	23572(753)
G502501	33	44	43	79.0	225(78)	43	1181.8	6183(771)
02	34	44	43	86.4	213(84)	43	1057.5	5558(1051)
03	34	44	44	114.0	243	44	2113.7	9732
04	34	42	42	54.4	125	42	671.7	3979
G504001	27	41	41	908.6	1317	41	3600	10817
02	27	41	40	487.9	850(60)	40	3600	11180(328)
03	27	42	41	606.3	1099(54)	41	3600	13011(232)
04	27	42	41	768.5	1304(96)	41	3600	12072(1490)
G506001	20	36	34	3600	3790(1572)	36	3600	5778(50)
02	20	35	35	3600	3513	36	3600	7032
03	20	36	35	3600	3856(202)	35	3600	7540(5829)
04	20	35	35	3600	3835	35	3600	6636
G602501	41	54	53	370.2	414(66)	53	3600	7045(289)
02	40	53	53	332.3	339	53	3600	7075
03	41	52	52	226.5	201	52	3565.4	8636
04	41	53	52	253.1	289(108)	52	3600	7380(1786)
G604001	32	51	50	3600	2392(90)	50	3600	4602(1066)
02	33	52	50	3600	2418(174)	50	3600	4383(2373)
03	32	50	49	2744	1770(62)	49	3600	5095(172)
04	32	50	50	3600	2321	50	3600	5075
G606001	25	45	45	3600	1392	45	3600	2380
02	23	46	44	3600	1593(156)	44	3600	3245(1583)
03	24	45	44	3600	1528(378)	45	3600	3303
04	24	46	45	3600	1523(64)	45	3600	2260(214)

The instances have an identification name of the type G_{xyz} , where x specifies the quantity of nodes of the graph, y the percentage of deleted nodes and, z is a sequential number generated for each test case (each parameter is specified using two digits).

It is important to observe that for the non-optimized branch-and-bound method, the solution obtained using the Smaller Cost Node heuristic was also used as the initial upper bound.

In Table 4, the columns are identified as follows: Id: instance identification; lb_c : lower bound value; NH: solution value given by the heuristic; P1: best solution value given by the optimized branch-and-bound; t1: computational time spent by optimized branch-and-bound; N1: total number of visited nodes in the optimized branch-and-bound (the amount of nodes visited until the best solution was found is indicated between parenthesis. When blank, means that the procedure did not find a better solution than the one determined by the heuristic); P2, t2, N2 are similar to P1, t1, N1, respectively, but applied to the non-optimized branch-and-bound.

As can be seen, the gain in computational time is expressive. For instance, in the example G302501, 70 nodes were visited and the best solution was found after visiting 36 nodes. In the non-optimized version, 663 nodes were visited and the best solution was found after visiting 143 nodes. The optimized procedure found the optimal solution around 7 times faster.

From the test performed, we can again verify that the quality of the solution obtained by the Smaller Cost Node heuristic was quite good. The quality of the lower bound, on the other hand, was quite poor.

7. Conclusion and future works

In this work, a new heuristic (Smaller Cost Node heuristic) to solve MOSP was presented. From the computational tests performed, the quality of the solution obtained by this heuristic was quite good.

In some industrial environments (like glass, wood or steel industries) solutions for the MOSP, close to the optimum can be acceptable. In these cases, solutions given by a heuristic can be utilized. In others environments, it is important to get an optimum solution, for instance, in the manufacture of some printed circuit boards, a reduction of a single line can represent an important saving on the industrial cost of the product, or even, become viable its fabrication. As showed in [16], to minimize the maximum number of lines on a particular type of VLSI circuit is equivalent to MOSP. This is one motivation for the search of more efficient exact methods for the MOSP, even though this is a NP-hard problem (see [16]). Regarding this application we performed some limited computational tests with instances having a considerable number of piece types and producing a large number of stacks.

In some practical industrial settings there might be a limitation on the maximum number of open stacks. The optimal sequencing of the patterns may not satisfy this limitation. In this case, the cutting patterns will also have to be looked for. This is a more general problem whose solution is quite challenging since it integrates two complex (NP-hard) problems, the cutting stock problem and the pattern sequencing problem. We cannot find much in the literature where this integrated problem is treated. Recently, in [18], an ILP formulation to this integrated problem was proposed and, in [19], heuristic methods to solve this integrated problem were suggested.

The computational complexity of the implemented procedures proposed in this work, given an instance with n pieces and m patterns, are $O(n^3)$ and $O((\min\{n, m\})!)$, for the calculation of lb_c and the branch-and-bound method, respectively. The interested reader may refer to [15] for the details of the calculations.

It is worth observing that in [17] an arc contraction heuristic to solve MOSP was presented. Computational tests performed have shown that the Smaller Cost Node Heuristic, proposed here, performs better than the contraction heuristic both in terms of quality and computational effort.

The computational tests performed and the results obtained indicate that there is room for research in the determination of good lower bounds for the MOSP. The proposed lower bound procedure provides quite poor results for certain instances of the MOSP.

There is still great interest in finding better lower bounds for the problem. The best bound presented in [17] and explained in this work does not provide good estimates for all instances of the problem.

Research is currently being developed to evaluate the use of node dominance (see [15]) in order to reduce the number of nodes to be enumerated in the branch-and bound method of [13,14].

Acknowledgements

This work was partially financed by grants from FAPESP and CNPq.

References

- [1] Yanasse HH. Minimization of open orders—polynomial algorithms for some special cases. *Pesquisa Operacional* 1996;16(1):1–26.
- [2] Dyson RG, Gregory AS. The cutting stock problem in the flat glass industry. *Operations Research Quarterly* 1974;25:41–54.
- [3] Madsen OBG. Glass cutting in a small firm. *Mathematical Programming* 1979;17:85–90.
- [4] Madsen OBG. An application of travelling-salesman routines to solve pattern-allocation problems in the glass industry. *Journal of Operational Research Society* 1988;39(3):249–56.
- [5] Lins S. Traversing trees and scheduling tasks for duplex corrugator machines. *Pesquisa Operacional* 1989;9:40–54.
- [6] Fink A, Voß S. Applications of modern heuristic search methods to pattern sequencing problems. *Computer and Operations Research* 1999;26(1):17–34.
- [7] Foerster HE, Wäscher G. Simulated annealing for order spread minimization in sequencing cutting patterns. *European Journal of Operational Research* 1998;110:272–81.
- [8] Yuen BJ. Heuristics for sequencing cutting patterns. *European Journal of Operational Research* 1991;55:183–90.
- [9] Yuen BJ. Improved heuristics for sequencing cutting patterns. *European Journal of Operational Research* 1995;87:57–64.
- [10] Faggioli E, Bentivoglio CA. Heuristic and exact methods for the cutting sequencing problem. *European Journal of Operational Research* 1998;110(3):564–75.
- [11] Yuen BJ, Richardson KV. Establishing the optimality of sequencing heuristics for cutting stock problems. *European Journal of Operational Research* 1995;84:590–8.
- [12] Yanasse HH. A transformation for solving a pattern sequencing problem in the wood cut industry. *Pesquisa Operacional* 1997;17(1):57–70.
- [13] Limeira MS. Development of an exact algorithm for solving a cutting pattern sequencing problem. Master, Brazilian Space Research Institute, São José dos Campos, Brazil, 1998 [in portuguese].
- [14] Yanasse HH. On a pattern sequencing problem to minimize the maximum number of open stacks. *European Journal of Operational Research* 1997;100:454–63.

- [15] Becceneri JC. The pattern sequencing problem to minimize the maximum number of open stacks in industrial cutting environments. Dissertation, Aeronautic Institute of Technology, Brazil, 1999 [in portuguese].
- [16] Linhares A, Yanasse HH. Connections between cutting-pattern sequencing, VLSI design, and flexible machines. *Computers and Operations Research* 2002;29(12):1759–72.
- [17] Yanasse HH, Becceneri JC, Soma NY. Bounds for a problem of sequencing patterns. *Pesquisa Operacional* 1999;19(2):249–77.
- [18] Yanasse HH, Pinto MJ. A cutting and sequencing integrated model. The Sixteenth Triennial Conference of the International Federation of Operational Research Societies, Edinburgh, Scotland, 2002. Programme, p. 121.
- [19] Pileggi GCF. Approaches for the optimization of the cutting patterns generation and sequencing integrated problems. Dissertation, University of São Paulo, Brazil, 2002 [in portuguese].